

Composante Blockchain

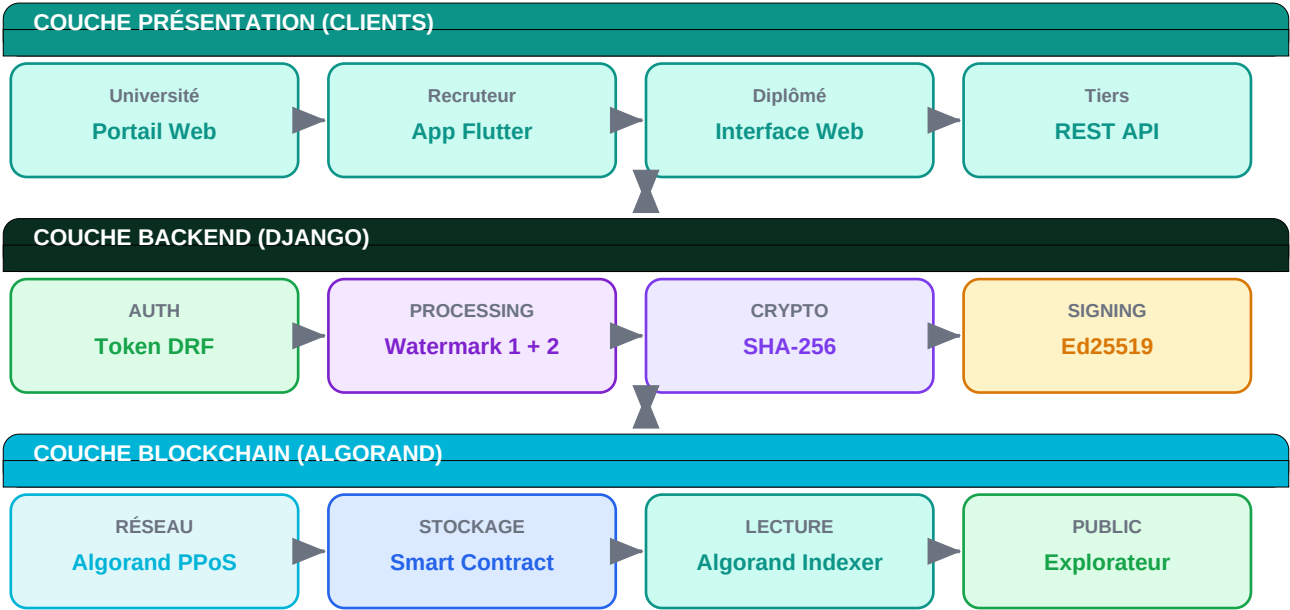
Certification Numérique des Diplômes — Burkina Faso

Stack : Algorand PPoS · Django · Ed25519 · SHA-256 · Flutter · Watermark double rôle

Principe clé : Le **Watermark** remplace le **QR Code**. Le PDF final contient deux filigranes : **Watermark 1** (métadonnées école/étudiant — appliqué avant le hash, garant de l'intégrité) et **Watermark 2** (TxID Algorand + hash + signature — appliqué après la transaction, vecteur de vérification). Le recruteur uploade simplement le PDF : tout est extrait automatiquement.

1. Architecture globale — 3 couches

Le système repose sur trois couches. La **couche Présentation** expose le portail web de l'université, l'**App Flutter** du recruteur et l'interface web du diplômé. La **couche Backend Django** orchestre l'authentification (**Token DRF**), l'application des deux watermarks, le calcul SHA-256 et la signature Ed25519. La **couche Blockchain Algorand** stocke les transactions via un **Smart Contract** et les rend lisibles publiquement via l'**Algorand Indexer**.



App Flutter

Recruteur — vérification

Token DRF

Auth API Django

Algorand PPoS

Réseau blockchain

Smart Contract

Stockage on-chain

2. Le double Watermark — Intégrité + Vérification

Le watermark joue deux rôles distincts et complémentaires dans le cycle de vie du diplôme. Le **Watermark 1** est un filigrane appliqué sur le PDF brut *avant* tout calcul cryptographique. Il encode les métadonnées de l'émission (nom de l'école, identité de l'étudiant, niveau, domaine, date, numéro interne). Puisque le hash SHA-256 est calculé sur ce PDF déjà watermarqué, WM1 fait partie intégrante de la preuve — le supprimer ou le modifier invalide immédiatement la signature. Le **Watermark 2** est appliqué *après* l'enregistrement on-chain : il insère dans le PDF le TxID Algorand, le hash et la signature, éliminant le besoin d'un QR Code séparé.

Watermark 1 — Rôle : Intégrité

Appliqué sur le PDF brut
Contient : nom école, étudiant,
niveau, domaine, date, ID interne

Le SHA-256 est calculé SUR ce PDF
→ **fait partie du hash on-chain**

Si WM1 supprimé ou modifié :
hash change → signature invalide

Watermark 2 — Rôle : Vérification

Appliqué APRÈS la transaction
Contient : TxID Algorand,
hash SHA-256, signature Ed25519

Remplace le QR Code :
tout est dans le PDF lui-même

Le recruteur uploade le PDF →
backend extrait WM2 pour vérifier

PyMuPDF

Insertion watermark PDF

reportlab

Génération filigrane

PyMuPDF

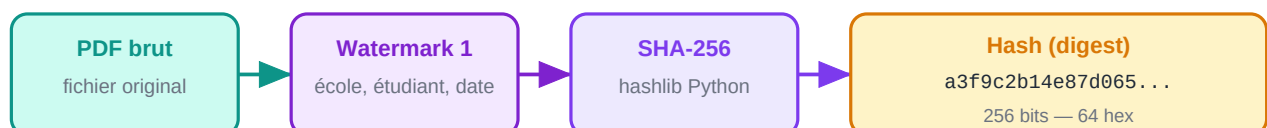
Extraction watermark

Django

Orchestration backend

3. Hachage SHA-256 — Empreinte du PDF + Watermark 1

Après application du Watermark 1, le module **Crypto** du backend Django calcule l'empreinte numérique du document via **SHA-256** (bibliothèque **hashlib** native Python). Cette fonction produit un digest de 256 bits (64 caractères hexadécimaux) — unique, déterministe et irréversible. L'**effet avalanche** garantit que modifier un seul octet du PDF (y compris le WM1) produit un hash radicalement différent. C'est ce hash qui sera signé par Ed25519 et stocké on-chain.



Important : le hash est calculé sur PDF + WM1 uniquement — WM2 n'est pas encore présent

hashlib

SHA-256 natif Python

PyMuPDF

Lecture PDF binaire

Django

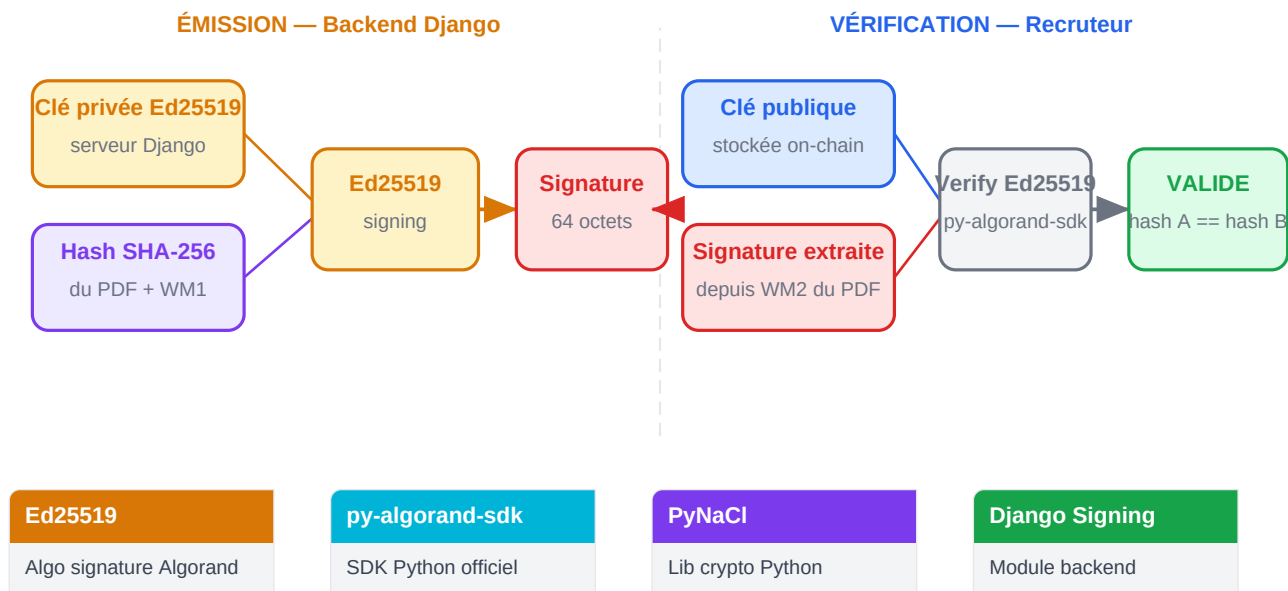
Module Crypto backend

SHA-256

256 bits — 64 hex

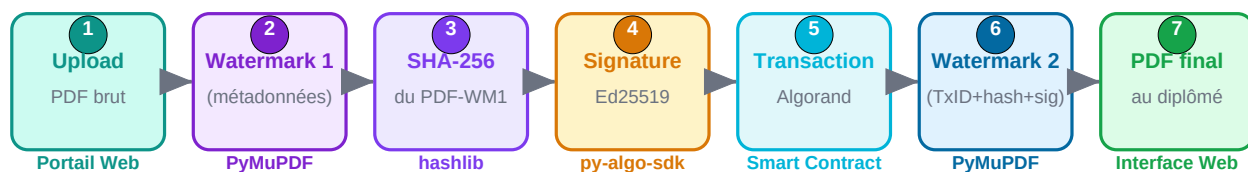
4. Signature numérique Ed25519 — Module Signing Django

Le module **Signing** utilise **Ed25519**, l'algorithme de signature natif d'Algorand (courbe Edwards Curve25519). Chaque établissement possède une **clé privée** conservée sur le serveur Django, qui signe le hash SHA-256 du PDF+WM1 pour produire une signature de 64 octets. La **clé publique** correspondante est enregistrée on-chain sur Algorand. Lors de la vérification, le backend extrait la signature depuis le **Watermark 2** du PDF présenté, puis la vérifie avec la clé publique récupérée via l'**Algorand Indexer** grâce au SDK officiel **py-algorand-sdk**.



5. Flux d'émission complet — Du PDF brut au PDF final

L'université uploade un PDF via le portail web. Le backend Django exécute automatiquement 7 étapes : (1) réception du PDF brut via l'API sécurisée **Token DRF** ; (2) application du **Watermark 1** (métadonnées) ; (3) calcul du **hash SHA-256** du PDF+WM1 ; (4) **signature Ed25519** du hash ; (5) envoi de la transaction au **Smart Contract Algorand** → obtention du TxID ; (6) application du **Watermark 2** (TxID + hash + signature) sur le PDF ; (7) PDF final transmis au diplômé. Aucun QR Code séparé n'est généré — le PDF est autoporteur.



Watermark 1 : ecole, etudiant, date Watermark 2 : TxID Algorand + hash + signature

Le hash est calculé sur le PDF après Watermark 1 — avant Watermark 2

Django REST

API émission Token DRF

PyMuPDF

WM1 + WM2 insertion

py-algorand-sdk

Transaction on-chain

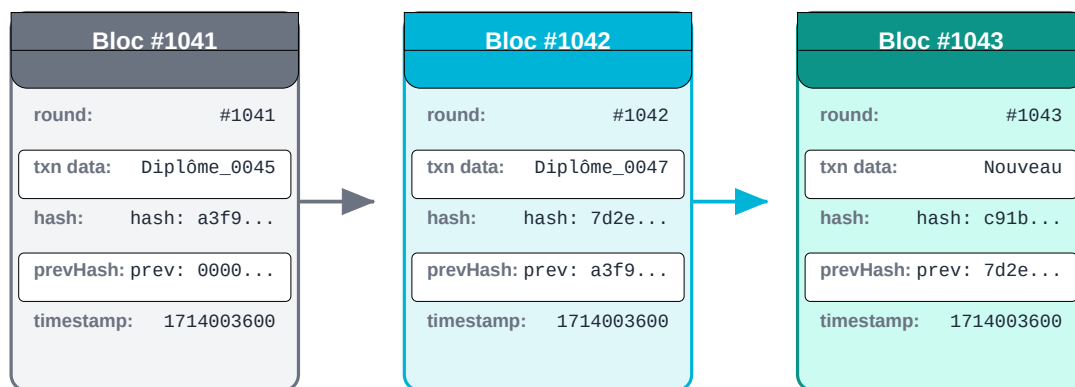
hashlib + Ed25519

Hash + Signature

6. Blockchain Algorand PPoS — Immuabilité et Smart Contract

Algorand utilise le protocole **Pure Proof of Stake (PPoS)** : les validateurs sont sélectionnés aléatoirement parmi tous les détenteurs d'ALGO, garantissant décentralisation et finalité en **~4 secondes**. Le **Smart Contract** (écrit en **PyTeal / ARC-4**) reçoit et stocke de façon permanente : le hash SHA-256, la signature Ed25519, la clé publique de l'établissement, le CID IPFS du PDF, et les métadonnées. Chaque bloc Algorand contient le hash du bloc précédent (**prevHash**) — toute modification rétroactive est détectée et rejetée par le réseau.

L'**Algorand Indexer** permet la lecture publique sans nœud complet.



PPoS : validateurs sélectionnés aléatoirement — finalité en ~4 secondes — modification rétroactive impossible

Algorand PPoS

Consensus ~4s

PyTeal / ARC-4

Smart Contract

Algorand Indexer

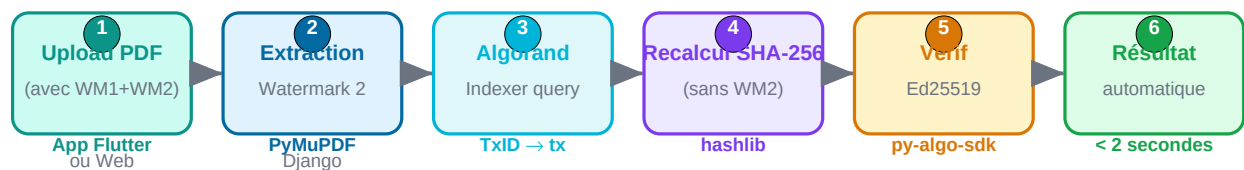
Lecture publique

IPFS / Pinata

Stockage PDF

7. Flux de vérification — Upload PDF → Résultat automatique

Le recruteur uploade simplement le PDF du diplôme sur le site ou l'**App Flutter**. Le backend Django exécute 6 étapes automatiques : (1) réception du PDF (contenant WM1 + WM2) ; (2) **extraction du Watermark 2** pour récupérer TxID, hash stocké et signature ; (3) interrogation de l'**Algorand Indexer** avec le TxID pour récupérer la transaction on-chain ; (4) recalcul du **SHA-256** du PDF après retrait temporaire du WM2 (le WM1 reste présent) pour retrouver le hash original ; (5) **vérification Ed25519** de la signature avec la clé publique de l'établissement ; (6) résultat affiché en moins de 2 secondes. Aucune interaction avec l'université n'est nécessaire.



Étape 4 : le recalcul SHA-256 exclut WM2 (retiré temporairement) pour retrouver le hash original

App Flutter

Upload + affichage

PyMuPDF

Extraction WM2

Algorand Indexer

Lecture transaction

py-algorand-sdk

Vérif Ed25519

Pourquoi c'est infalsifiable

Tentative de fraude	Mécanisme de défense	Technologie
Modifier le PDF après certification	Hash recalculé ≠ hash on-chain Algorand	SHA-256 + Smart Contract
Supprimer le Watermark 1	Hash change → signature Ed25519 invalide	SHA-256 + Ed25519
Falsifier le Watermark 2 (faux TxID)	TxID inexistant sur Algorand Indexer → rejeté	Algorand Indexer
Usurper l'identité d'un établissement	Impossible sans la clé privée Ed25519 de l'école	Ed25519 + py-algorand-sdk
Modifier une transaction on-chain	prevHash brisé → réseau PPOS rejette la chaîne modifiée	Algorand PPOS

Stack technologique complète

Présentation	Portail Web React.js — Université	App Flutter Recruteur — vérification	Interface Web Diplômé — consultation	REST API Intégration tiers
Backend Django	Django Framework Python	Token DRF Auth API sécurisée	PyMuPDF WM1 + WM2 PDF	hashlib SHA-256 natif
Cryptographie	Ed25519 Signature numérique	py-algorand-sdk SDK Algorand Python	PyNaCl Lib crypto Python	SHA-256 Hachage document

Blockchain	Algorand PPoS Réseau consensus	PyTeal / ARC-4 Smart Contracts	Algo Indexer Lecture on-chain	IPFS / Pinata Stockage PDF
-------------------	--	--	---	--------------------------------------

Note déploiement : Phase MVP sur **Algorand Testnet** (gratuit, identique au Mainnet). Production sur **Algorand Mainnet** — coût par transaction : ~0.001 ALGO (~0.0002 USD). Le PDF final est **autoporteur** : WM1 garantit l'intégrité, WM2 contient toutes les données de vérification.